Dataset overview

PlantVillage (training) and PlantDoc (field test). Every number on these pages was measured by `scripts/dataset_stats.py` on the real data, or read from our code.

1. What is PlantVillage?

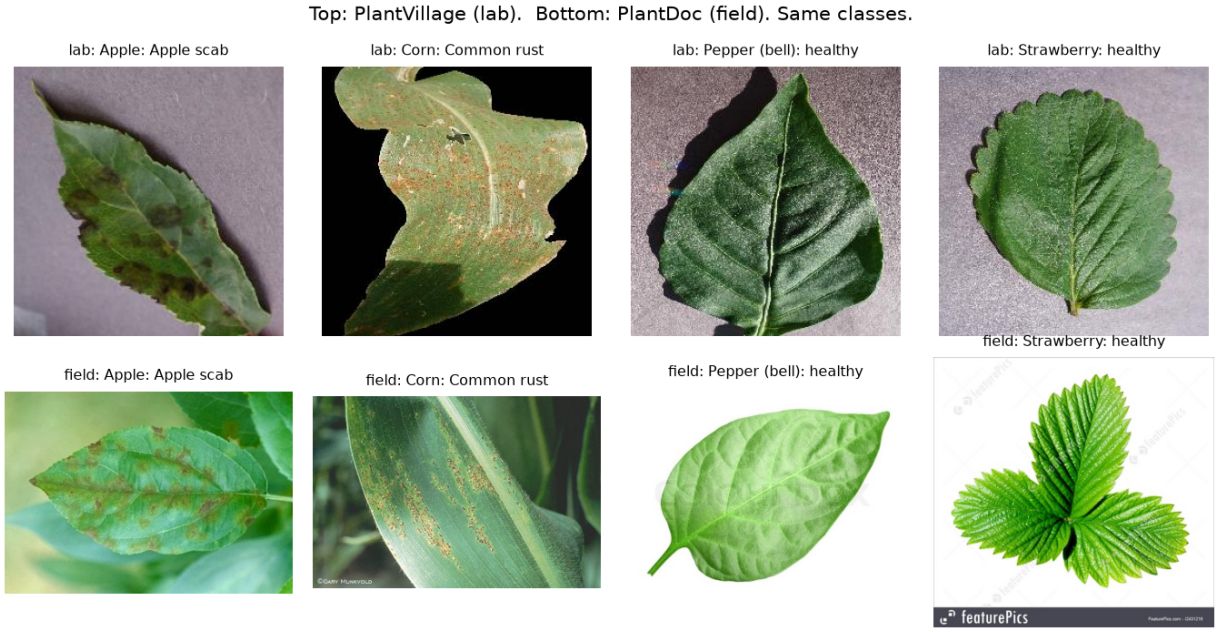
PlantVillage is an open collection of photos of single plant leaves, each labelled with the crop and the disease (or 'healthy'). The full colour version has **54,305 images** in **38 classes** from **14 crops**. 12 of the classes are 'healthy', the other 26 are diseases.

Who made it: the PlantVillage project (Hughes & Salathé, 2015). It became well known through the 2016 paper by Mohanty, Hughes and Salathé, 'Using deep learning for image-based plant disease detection'. We download the authors' own upload on Hugging Face (`mohanty/PlantVillage`): one 2.2 GB `data.zip` containing three versions of every photo (color, grayscale, segmented), plus the official train/test split files and a 'leaf map' that says which photos show the same physical leaf.

How the images were collected (from the dataset paper, not from our code): leaves were picked from the plant, laid on a plain sheet, and photographed one at a time. That is why every image shows one centred leaf on a plain grey background. All photos are 256x256 pixels and RGB (checked on the 76 training images prepared on this laptop).

Why the plain background matters

A plain, similar background makes the dataset easy: the model can find the leaf trivially, and accuracy on PlantVillage itself is usually very high. But a real farmer's photo has soil, other leaves, hands, shadows and uneven light. A model that only ever saw plain backgrounds may (a) never have learned to ignore clutter, and (b) secretly use the background as a clue. This is why our project includes the PlantDoc field test (domain shift) and the background-bias experiment.



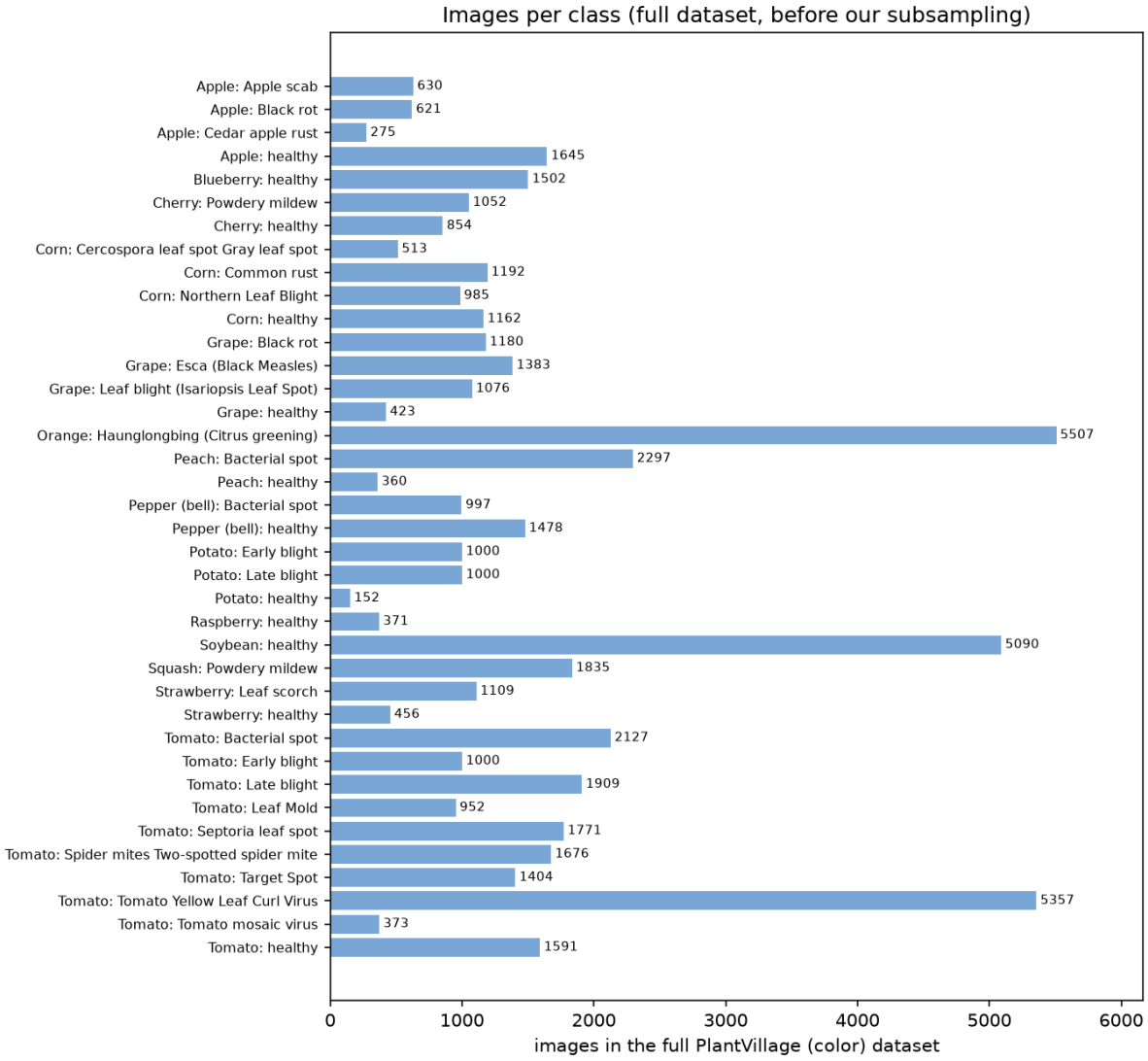
Real images from our prepared data folders.

2. Classes and crops

The 14 crops: Apple, Blueberry, Cherry (including sour), Corn (maize), Grape, Orange, Peach, Pepper, bell, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato.

#	Crop	Disease / healthy	Full dataset	Our train	Our val	Our test
0	Apple	Apple scab	630	200	40	60
1	Apple	Black rot	621	200	40	60
2	Apple	Cedar apple rust	275	183	40	52
3	Apple	healthy	1645	200	40	60
4	Blueberry	healthy	1502	200	40	60
5	Cherry (including sour)	Powdery mildew	1052	200	40	60
6	Cherry (including sour)	healthy	854	200	40	60
7	Corn (maize)	Cercospora leaf spot Gray leaf spot	513	200	40	60
8	Corn (maize)	Common rust	1192	200	40	60
9	Corn (maize)	Northern Leaf Blight	985	200	40	60
10	Corn (maize)	healthy	1162	200	40	60
11	Grape	Black rot	1180	200	40	60
12	Grape	Esca (Black Measles)	1383	200	40	60
13	Grape	Leaf blight (Isariopsis Leaf Spot)	1076	200	40	60
14	Grape	healthy	423	200	40	60
15	Orange	Haunglongbing (Citrus greening)	5507	200	40	60
16	Peach	Bacterial spot	2297	200	40	60
17	Peach	healthy	360	200	40	60
18	Pepper, bell	Bacterial spot	997	200	40	60
19	Pepper, bell	healthy	1478	200	40	60
20	Potato	Early blight	1000	200	40	60
21	Potato	Late blight	1000	200	40	60
22	Potato	healthy	152	80	40	32
23	Raspberry	healthy	371	167	40	60
24	Soybean	healthy	5090	200	40	60
25	Squash	Powdery mildew	1835	200	40	60
26	Strawberry	Leaf scorch	1109	200	40	60
27	Strawberry	healthy	456	200	40	60
28	Tomato	Bacterial spot	2127	200	40	60
29	Tomato	Early blight	1000	200	40	60
30	Tomato	Late blight	1909	200	40	60
31	Tomato	Leaf Mold	952	200	40	60
32	Tomato	Septoria leaf spot	1771	200	40	60
33	Tomato	Spider mites Two-spotted spider mite	1676	200	40	60
34	Tomato	Target Spot	1404	200	40	60
35	Tomato	Tomato Yellow Leaf Curl Virus	5357	200	40	60
36	Tomato	Tomato mosaic virus	373	200	40	60
37	Tomato	healthy	1591	200	40	60
	TOTAL		54,305	7,430	1,520	2,244

The '#' column is the label number the model uses: the position in `PV_CLASSES` (src/data.py).

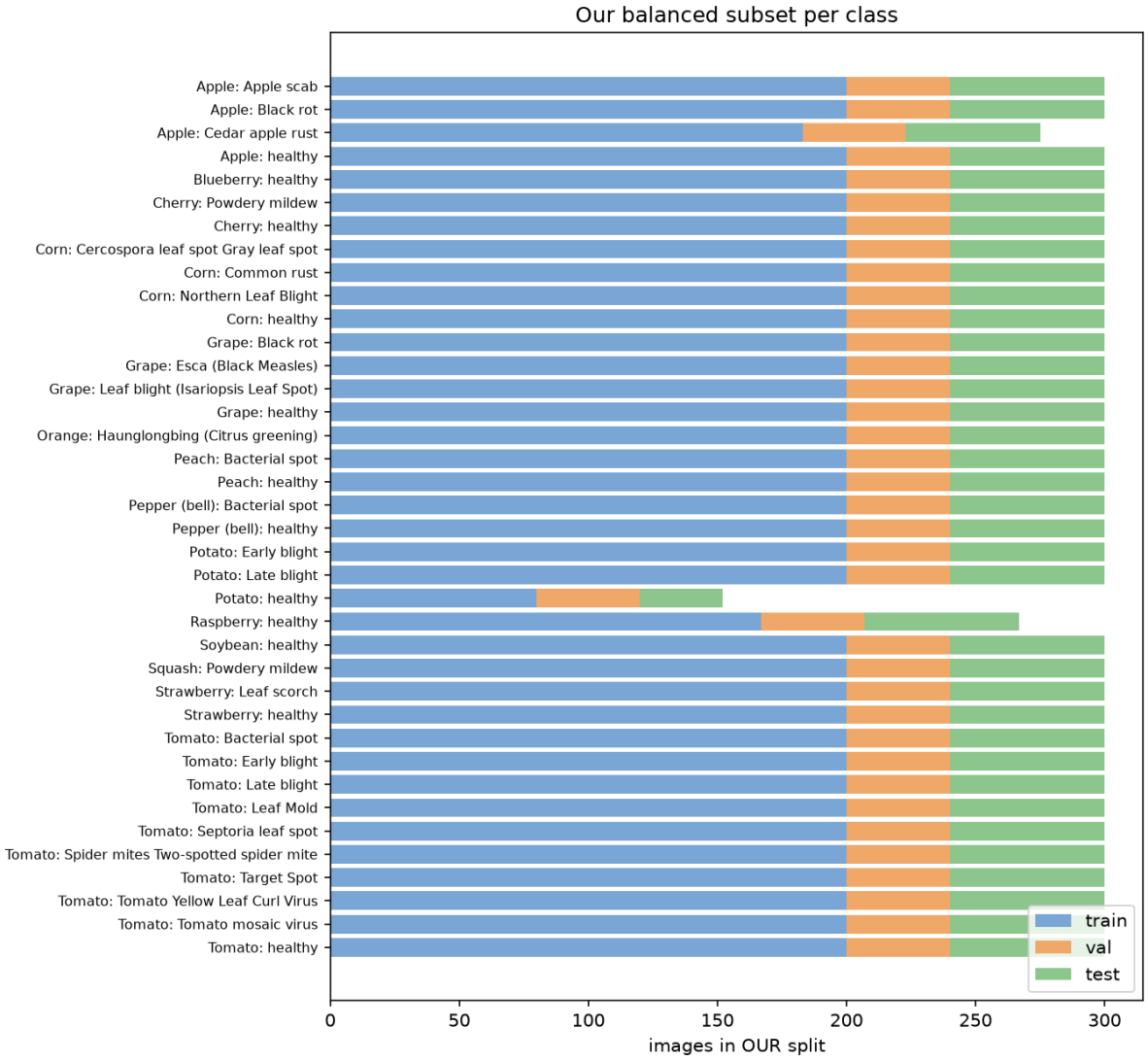


Images per class in the full dataset (from the zip's file list).

Class imbalance

The full dataset is very unbalanced: the largest class is **Orange: Haunglongbing (Citrus greening)** with **5,507** images, the smallest is **Potato: healthy** with **152** images (a ratio of about 36 : 1). If we trained on everything, the model would see Orange HLB 36 times more often than healthy potato, and accuracy would be dominated by the big classes.

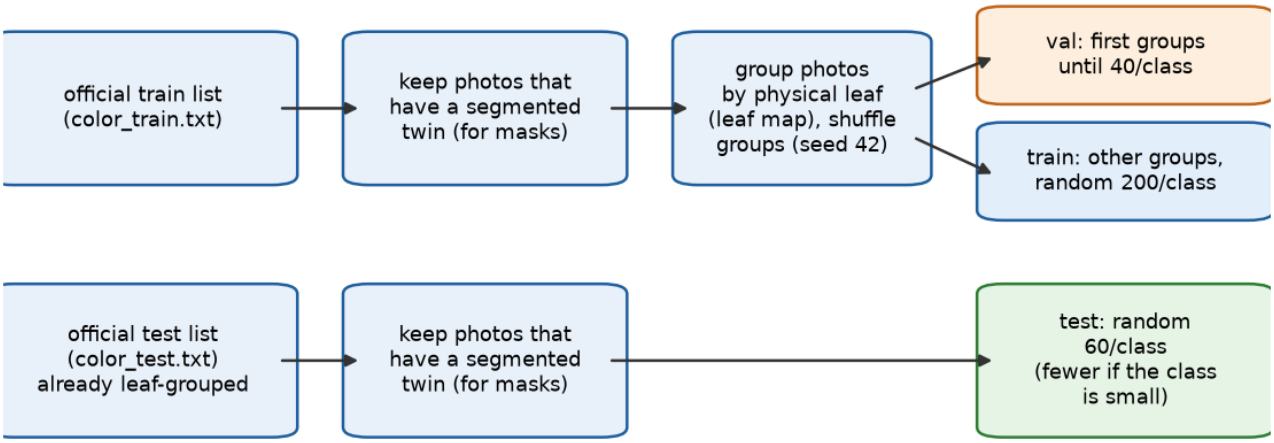
Our fix: we take at most 200 train, 40 val and 60 test images per class. Our subset is almost balanced: the largest class has 300 images in total, the smallest 152 (Potato: healthy, which simply has no more photos). Classes with fewer test images: Apple: Cedar apple rust (52), Potato: healthy (32). We also report macro-F1, which counts every class equally.



Train / val / test images per class in our subset.

3. How we split the data (and avoid leakage)

Leakage means information from the test set sneaks into training. In PlantVillage, the same physical leaf was often photographed several times. If photo 1 of a leaf is in train and photo 2 in test, the model is tested on an image it has practically seen, and test accuracy is too optimistic.



How plan_plantvillage_split (src/data.py) chooses the files.

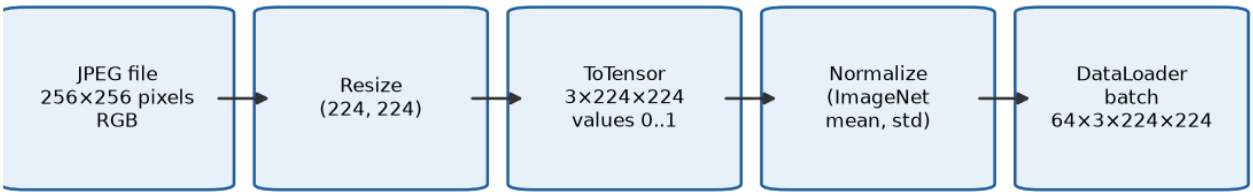
- **Test** = random sample (seed 42) from the authors' official test list, which they made so that no leaf is on both sides.
- **Train / val** come from the official train list. We group photos by leaf (using the leaf map, see `_leaf_group`), shuffle the groups, and give whole groups to val until it has 40 images per class. The other groups go to train (random 200 per class).
- Only photos that have a 'segmented' twin are used (we need it for the leaf mask). In practice that is all 54,305 colour photos.

Check (leaf groups shared between ...)	Result
train and val	0
val and test	0
train and test	2 (comes from the official split, which we use as it is)
number of leaf groups: train / val / test	5,225 / 671 / 1,507

Split	Images	Used for
train	7,430	updating the weights
val	1,520	choosing the best epoch, choosing the rejection threshold
test (PlantVillage)	2,244	final in-domain numbers, used only once at the end
PlantDoc (field)	2,578 in the repo	final out-of-domain numbers, never trained on

4. Images are tensors, not rows and columns

A table (like a CSV) has rows = examples and columns = features. Images do not fit that picture well: the 'features' are pixels, and their **position** matters. A pixel only means something together with its neighbours. So we keep the 2-D layout and store each image as a 3-D array (a tensor).



From file to model input (get_transforms('eval') in src/data.py).

Real numbers from our code: one image becomes a tensor of shape **3×224×224** (channels × height × width), type float32. After normalisation the values in this example range from -1.93 to 1.75 (about 0 on average). A batch stacks 64 images: **64×3×224×224**.

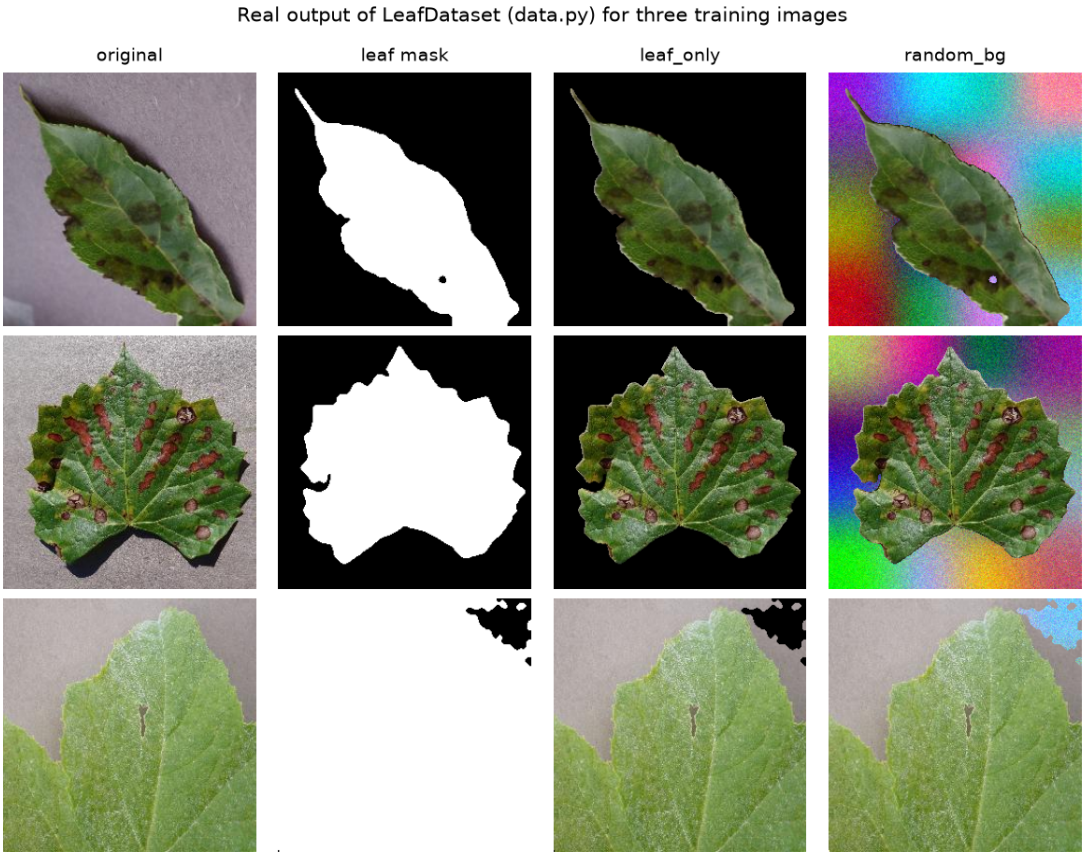
Small example. A tiny 2×2 RGB image is three 2×2 grids (one red, one green, one blue), so its shape is 3×2×2 = 12 numbers. Our 224×224 images have 3 × 224 × 224 = 150,528 numbers each. A conv layer slides small filters over these grids, so it can use the neighbourhood of every pixel. A row-and-column model would lose that.

Normalisation: `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` subtracts the ImageNet average colour and divides by its spread, channel by channel. The pretrained model was trained on inputs like that; we use the same for the scratch CNN so both models get identical inputs.

5. Leaf masks and backgrounds

The zip's 'segmented' version shows the same leaf with the background painted black. `make_leaf_mask` turns it into a mask: a pixel is 'leaf' if its brightest colour channel is above 25, then a 5×5 median filter removes speckles. The mask is saved as a PNG next to each image (`data/plantvillage/masks/...`).

On the 152 masks prepared on this laptop, the leaf covers on average **49%** of the image (smallest 13%, largest 100%).



Masks and the three background versions used in the background-bias experiment (real output of LeafDataset).

6. PlantDoc (field images)

PlantDoc (Singh et al., 2020) contains photos from the internet and the field: cluttered backgrounds, several leaves, hands, different light. The GitHub repository (pinned commit) has **28 class folders**, **2,342** images in its train folder and **236** in its test folder, **2,578** in total. We never train on it: we use both folders together as one field test set. `prepare_plantdoc` shrinks every photo to at most 512 px on the longer side and saves it under the PlantVillage class name.

Overlap: all 28 PlantDoc folders have a matching PlantVillage class (`PLANTDOC_TO_PLANTVILLAGE` in src/data.py), so there are **28 shared classes**. The other 10 PlantVillage classes never appear in PlantDoc.

PlantDoc folder	PlantVillage class	train	test
Apple Scab Leaf	Apple: Apple scab	83	10
Apple rust leaf	Apple: Cedar apple rust	79	10
Apple leaf	Apple: healthy	82	9
Blueberry leaf	Blueberry: healthy	106	11
Cherry leaf	Cherry: healthy	47	10
Corn Gray leaf spot	Corn: Cercospora leaf spot Gray leaf spot	64	4
Corn rust leaf	Corn: Common rust	106	10
Corn leaf blight	Corn: Northern Leaf Blight	180	12
grape leaf black rot	Grape: Black rot	56	8
grape leaf	Grape: healthy	57	12
Peach leaf	Peach: healthy	103	9

PlantDoc folder	PlantVillage class	train	test
Bell_pepper leaf spot	Pepper (bell): Bacterial spot	62	9
Bell_pepper leaf	Pepper (bell): healthy	53	8
Potato leaf early blight	Potato: Early blight	109	8
Potato leaf late blight	Potato: Late blight	97	8
Raspberry leaf	Raspberry: healthy	112	7
Soyabean leaf	Soybean: healthy	57	8
Squash Powdery mildew leaf	Squash: Powdery mildew	124	6
Strawberry leaf	Strawberry: healthy	88	8
Tomato leaf bacterial spot	Tomato: Bacterial spot	101	9
Tomato Early blight leaf	Tomato: Early blight	79	9
Tomato leaf late blight	Tomato: Late blight	101	10
Tomato mold leaf	Tomato: Leaf Mold	85	6
Tomato Septoria leaf spot	Tomato: Septoria leaf spot	140	11
Tomato two spotted spider mites leaf	Tomato: Spider mites Two-spotted spider mite	2	0
Tomato leaf yellow virus	Tomato: Tomato Yellow Leaf Curl Virus	70	6
Tomato leaf mosaic virus	Tomato: Tomato mosaic virus	44	10
Tomato leaf	Tomato: healthy	55	8

PlantVillage classes **not** in PlantDoc: Apple: Black rot, Cherry: Powdery mildew, Corn: healthy, Grape: Esca (Black Measles), Grape: Leaf blight (Isariopsis Leaf Spot), Orange: Haunglongbing (Citrus greening), Peach: Bacterial spot, Potato: healthy, Strawberry: Leaf scorch, Tomato: Target Spot.

Image sizes after our resizing vary (most common in the 54 images on this laptop: 512x384 (8), 429x512 (3), 512x288 (2), 288x512 (2)). They are not square, unlike PlantVillage.

Numbers to memorize

1. PlantVillage: 54,305 colour images, 38 classes, 14 crops (12 healthy classes).
2. Imbalance in the full data: Orange: Haunglongbing (Citrus greening) 5,507 vs Potato: healthy 152 images.
3. Our subset: 7,430 train / 1,520 val / 2,244 test (max 200/40/60 per class), seed 42.
4. Train/val are split by physical leaf; test is the authors' official leaf-grouped test split.
5. PlantVillage images: 256×256 RGB, one leaf, plain background.
6. Model input: 3×224×224 tensor; a batch is 64×3×224×224.
7. PlantDoc: 2,578 field photos (2,342 + 236), 28 folders, never used for training.
8. 28 classes are shared by both datasets; 10 PlantVillage classes are not in PlantDoc.
9. Leaf mask = brightest channel > 25 in the segmented image; leaf covers about 49% of an image on average.
10. ImageNet normalisation: mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225).

Things to double-check before the viva

These are places where the code is easy to misdescribe, or where something is limited or undocumented. Better to know them than to be surprised.

- The files now in `results/` come from a 1-epoch CPU smoke test on 2 images per class. They are NOT results. This builder detects that and prints [pending]. Run the Colab notebook, copy `results/` back, then run `python scripts/build_viva_pdfs.py` again.
- README.md says "see 'Getting it onto Colab' below", but that section does not exist in the README.

- Training on the command line (`python -m src.train`) uses batch size 32 and lr 1e-3 for BOTH models by default. The notebook uses batch 64, lr 0.001 (CNN) and 0.0005 (EfficientNet). Always quote the notebook values; those produced the results.
- The eval transform is `Resize((224, 224))`, which squashes non-square photos. PlantVillage photos are square (256×256), but PlantDoc photos are not (e.g. 512×384), so they get distorted. This is part of the measured domain gap.
- Leakage: our train and val share 0 leaf groups, but train and test share 2 leaf groups. These come from the authors' official split, which we use as is. Also, photos missing from the leaf map count as their own group, so the check cannot be 100% complete.
- PlantDoc's spider-mites folder has only 2 images, and several PlantDoc classes are small, so per-class field numbers are noisy. The PlantVillage test set has at most 60 images per class (Potato healthy 32, Apple cedar rust 52).
- Each experiment is run once with seed 42. Differences of 1-2 points between models may be noise. We report no confidence intervals or repeated runs.
- How PlantVillage was collected comes from the dataset paper (Hughes & Salathé 2015; Mohanty et al. 2016), not from our code. Check the wording against the paper before quoting details.
- The "~40 min on a T4" estimate is not measured yet. Read `train_time_sec` in `results/history_*.json` after the run.